

Master Module

Departments · Designations · Job Titles

Frontend API Implementation Guide

Date: 2026-09-21

Base paths: `/api/v1/departments` · `/api/v1/designations` · `/api/v1/job-titles` · `/api/v1/lookups`

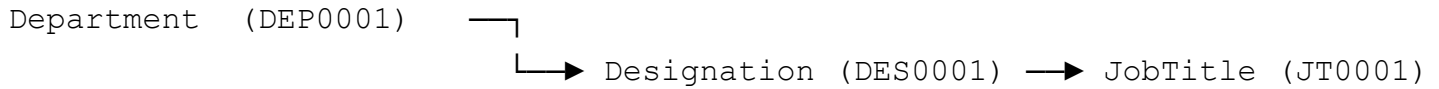
Source of truth: the actual backend code — `DepartmentController` , `DesignationController` , `JobTitleController` , `LookupController` , their request/response DTOs, their `*ServiceImpl` classes, `GlobalExceptionHandler` , `ApiError` , and the JWT entry-point/access-denied handlers. Nothing here is invented; every message, default and status code below was read out of the code. Anything the backend does **not** support is explicitly marked **"Not supported in backend"**.

At a Glance

Area	Details
Resources	Department → Designation → Job Title
Base paths	<code>/api/v1/departments</code> · <code>/api/v1/designations</code> · <code>/api/v1/job-titles</code> · <code>/api/v1/lookups</code>
Management roles	<code>HR_ADMIN</code> , <code>MANAGER</code>
Lookup access	Any authenticated user
Pagination	0-based, default <code>page=0</code> , <code>size=10</code>
Status change	<code>PATCH .../status?active=<boolean></code>
Primary recommendation	Prefer Deactivate over Delete for referenced records

1. What this module is

Three flat, independent reference ("master") tables, plus lookup endpoints that feed cascading dropdowns:



- A **Department** has no parent.
 - A **Designation** belongs to exactly one Department (`departmentId` required).
 - A **JobTitle** belongs to exactly one Designation (`designationId` required).
 - Codes are generated **server-side** and are read-only: `DEP0001` , `DES0001` , `JT0001` padded to 4 digits.
 - Deleting a parent is a **hard delete with no dependency check in the service layer** — see §11.2, this is the single biggest frontend hazard in this module.
 - Employee records reference all three of these tables with `NOT NULL` foreign keys.
-

2. Global conventions

2.1 Success envelope (`ApiResponse<T>`)

Every success response in this module is wrapped:

```
{
  "success": true,
  "message": "Department fetched successfully.",
  "data": { },
  "timestamp": "2026-09-21T11:04:22.918233"
}
```

`data` is `null` for delete and status-change endpoints (their generic type is `Void`).

2.2 Error envelope (`ApiError`)

```
{
  "success": false,
  "message": "Department not found.",
  "errorCode": "RESOURCE_NOT_FOUND",
}
```

```

"path": "/api/v1/departments/999",
"timestamp": "2026-09-21T11:04:22.918233"
}

```

`errorCode` is an enum name (string). Codes you will actually see in this module:

HTTP	errorCode	Emitted by	Message
400	VALIDATION_FAILED	@Valid bean validation failures	the DTO's message, e.g. "Job title is required." ; for the master DTOs without a custom message the fallback is "must not be blank" / "must not be null"
400	VALIDATION_FAILED	Duplicate name checks in the service	"Department already exists." / "Designation already exists in this department." / "Job title already exists."
400	INVALID_REQUEST	malformed JSON / wrong scalar type / bad query-param type	"Invalid request format", "Invalid value for field: X", or "Invalid value 'abc' for parameter 'active'"
401	UNAUTHORIZED	JwtAuthenticationEntryPoint	"Authentication is required to access this resource."
403	ACCESS_DENIED	JwtAccessDeniedHandler / GlobalExceptionHandler	"You do not have permission to perform this action." / "Access denied."
404	RESOURCE_NOT_FOUND	ResourceNotFoundException	"Department not found." / "Designation not found." / "Job title not found."
500	INTERNAL_SERVER_ERROR	catch-all handler	"An unexpected error occurred. (...) " – used for FK violations, missing required query params, unknown <code>sortBy</code> , etc.

Note: `ApiError` has **no `errors[]` array**. For a multi-field validation failure the backend returns **only the first** field error message. Do all field-level validation client-side; never rely on

the server to list every problem.

2.3 Authentication & roles

- All master endpoints require a Bearer JWT – send `Authorization: Bearer <accessToken>`.
- The **effective** allowed roles are `HR_ADMIN` and `MANAGER` only. The Swagger `@Tag` on the controllers claims `SUPER_ADMIN` is allowed, **but the `@PreAuthorize` expressions do not include it** – a `SUPER_ADMIN` user gets **403**. Treat the Swagger tag as wrong.
- `PAYROLL_ADMIN`, `EMPLOYEE`, `CLIENT` → 403 on every endpoint in this module.
- The lookup endpoints (§7) only require `isAuthenticated()`, i.e. **any logged-in user**.

2.4 Paging / search / sorting (list endpoints only)

All three list endpoints share the exact same contract:

Query param	Type	Default	Notes
<code>page</code>	int	0	0-based . There is no server-side max <code>size</code> .
<code>size</code>	int	10	
<code>search</code>	string	""	Optional. Blank/absent → return everything.
<code>sortBy</code>	string	<code>departmentName</code> / <code>designationName</code> / <code>jobTitle</code>	Must be a JPA entity property name , not a display label (see the allowlist in §11.4). An unknown value → 500 .
<code>sortDirection</code>	string	<code>asc</code>	Only <code>desc</code> (case-insensitive) is special-cased. Anything else silently means ascending – <code>"DESC"</code> , <code>"down"</code> , <code>""</code> , typos all yield ascending.

- `search` does a **case-insensitive CONTAINS on the name field only** – never on the code, never on the description.
- Paging is done in SQL; the response is a `PageResponse<T>` :

```
{
  "content": [ ],
  "page": 0,
  "size": 10,
```

```
"totalElements": 27,  
"totalPages": 3,  
"first": true,  
"last": false  
}
```

`first` and `last` are always present in the payload (they are primitive booleans), so they are usable directly.

2.5 Field length limits (enforced by the DB, not by bean validation)

Exceeding these does not produce a 400 – it produces a **500** from the DB. Validate on the client:

Field	Max length	DB column
<code>departmentName</code>	100	<code>departments.department_name</code>
<code>description</code> (department / designation)	255	<code>...description</code>
<code>departmentCode</code> / <code>designationCode</code> / <code>jobTitleCode</code>	20 / 255 / 255	server-generated, read-only
<code>designationName</code>	255	<code>designations.designation_name</code>
<code>jobTitle</code>	100	<code>job_titles.job_title</code>

2.6 No update-on-create, no bulk, no export

There is **no** bulk create, bulk delete, CSV import, or export endpoint in this module. Not supported in backend.

3. API list

Departments – `/api/v1/departments`

#	Method	Endpoint	Purpose	Success	Roles
1	POST	<code>/api/v1/departments</code>	Create department	201	HR_ADMIN, MANAGER
2	GET	<code>/api/v1/departments/{id}</code>	Get one	200	HR_ADMIN, MANAGER
3	GET	<code>/api/v1/departments</code>	Paged list + search	200	HR_ADMIN, MANAGER
4	PUT	<code>/api/v1/departments/{id}</code>	Full update	200	HR_ADMIN, MANAGER
5	PATCH	<code>/api/v1/departments/{id}/status?active=true</code>	Activate / deactivate	200	HR_ADMIN, MANAGER
6	DELETE	<code>/api/v1/departments/{id}</code>	Hard delete	200	HR_ADMIN, MANAGER

Designations — `/api/v1/designations`

#	Method	Endpoint	Purpose	Success	Roles
7	POST	<code>/api/v1/designations</code>	Create designation	201	HR_ADMIN, MANAGER
8	GET	<code>/api/v1/designations/{id}</code>	Get one	200	HR_ADMIN, MANAGER
9	GET	<code>/api/v1/designations</code>	Paged list + search	200	HR_ADMIN, MANAGER
10	PUT	<code>/api/v1/designations/{id}</code>	Full update	200	HR_ADMIN, MANAGER
11	PATCH	<code>/api/v1/designations/{id}/status?active=false</code>	Activate / deactivate	200	HR_ADMIN, MANAGER
12	DELETE	<code>/api/v1/designations/{id}</code>	Hard delete	200	HR_ADMIN, MANAGER

Job titles — `/api/v1/job-titles`

#	Method	Endpoint	Purpose	Success	Roles
13	POST	<code>/api/v1/job-titles</code>	Create job title	201	HR_ADMIN, MANAGER
14	GET	<code>/api/v1/job-titles/{id}</code>	Get one	200	HR_ADMIN, MANAGER
15	GET	<code>/api/v1/job-titles</code>	Paged list + search	200	HR_ADMIN, MANAGER
16	PUT	<code>/api/v1/job-titles/{id}</code>	Full update	200	HR_ADMIN, MANAGER
17	PATCH	<code>/api/v1/job-titles/{id}/status?active=true</code>	Activate / deactivate	200	HR_ADMIN, MANAGER
18	DELETE	<code>/api/v1/job-titles/{id}</code>	Hard delete	200	HR_ADMIN, MANAGER

Lookups (dependency dropdowns) – `/api/v1/lookups`

#	Method	Endpoint	Returns	Roles
19	GET	<code>/api/v1/lookups/departments</code>	Active departments	any authenticated user
20	GET	<code>/api/v1/lookups/departments/{departmentId}/designations?departmentId={id}</code>	Active designations of a department	any authenticated user
21	GET	<code>/api/v1/lookups/designations/{designationId}/job-titles?designationId={id}</code>	Active job titles of a designation	any authenticated user

Endpoints 20 and 21 **also need the value as a query parameter** – see §11.1. This is a backend defect you must work around, not a typo in this document.

4. Payload reference

4.1 Request DTOs

CreateDepartmentRequest → POST /api/v1/departments

Field	JSON type	Required	Validation	Notes
departmentName	string	yes	@NotBlank (default message "must not be blank")	trimmed? no – the server does not trim ; trim client-side
description	string	no	none	max 255 (DB)
active	—	—	—	not accepted ; new rows are always <code>active = true</code>
departmentCode	—	—	—	not accepted ; auto-generated

UpdateDepartmentRequest → PUT /api/v1/departments/{id}

Field	JSON type	Required	Validation	Notes
departmentName	string	yes	@NotBlank	
description	string	no	none	
active	boolean	yes, in practice	none	DANGER: it is a primitive <code>boolean</code> . If you omit it, Jackson coerces it to <code>false</code> and the department is silently deactivated . Always send the current value back on update.

CreateDesignationRequest → POST /api/v1/designations

Field	JSON type	Required	Validation
designationName	string	yes	@NotBlank
departmentId	number(int64)	yes	@NotNull
description	string	no	none

UpdateDesignationRequest → PUT /api/v1/designations/{id}

Field	JSON type	Required	Validation	Notes
designationName	string	yes	@NotBlank	
departmentId	number	yes	@NotNull	changing it re-parents the designation
description	string	no	none	
active	boolean	yes, in practice	none	same primitive-boolean trap as above

CreateJobTitleRequest → POST /api/v1/job-titles

Field	JSON type	Required	Validation
jobTitle	string	yes	@NotBlank(message = "Job title is required.")
designationId	number	yes	@NotNull(message = "Designation is required.")

UpdateJobTitleRequest → PUT /api/v1/job-titles/{id}

Field	JSON type	Required	Validation	Notes
jobTitle	string	yes	@NotBlank(message = "Job title is required.")	

Field	JSON type	Required	Validation	Notes
<code>designationId</code>	number	yes	<code>@NotNull(message = "Designation is required.")</code>	
<code>active</code>	boolean	yes, in practice	none	same primitive-boolean trap

`PUT` is a **full replace**, not a partial patch. There is no `PATCH` for individual fields – only the `/status` sub-resource.

4.2 Response DTOs

DepartmentResponse

```
{
  "id": 4,
  "departmentCode": "DEP0004",
  "departmentName": "Information Technology",
  "description": "Software engineering and IT operations",
  "active": true
}
```

DesignationResponse

```
{
  "id": 12,
  "designationCode": "DES0012",
  "designationName": "Software Engineer",
  "departmentId": 4,
  "departmentName": "Information Technology",
  "description": null,
  "active": true
}
```

JobTitleResponse

```
{
  "id": 31,
```

```
"jobTitleCode": "JT0031",
"jobTitle": "Backend Engineer",
"designationId": 12,
"designationName": "Software Engineer",
"active": true
}
```

Lookup DTOs

```
// GET /api/v1/lookups/departments (also for designations / job-titles)
[ { "id": 4, "name": "Information Technology" } ]
```

Note the lookup DTO is **generic**: it always uses `name`, so a designation lookup returns `{"id":12,"name":"Software Engineer"}` – no `departmentId` echo, no code.

Not exposed anywhere in this module: `createdAt`, `updatedAt`, and the lookup DTOs do not carry the parent id. Not supported in backend.

5. Endpoint-by-endpoint details

5.1 Create Department

Item	Value
Method / path	<code>POST /api/v1/departments</code>
Roles	HR_ADMIN, MANAGER
Request	<code>CreateDepartmentRequest</code> (§4.1)
Response	201+ envelope, <code>data = DepartmentResponse</code>
Message	<code>"Department created successfully."</code>

Server behaviour: rejects if `existsByDepartmentName(departmentName)` (exact, case-sensitive match – see §11.3) with 400 `VALIDATION_FAILED "Department already exists."`. Otherwise generates the next code from the highest existing `departmentCode`, sets `active = true`, saves.

POST /api/v1/departments

```
{ "departmentName": "Finance", "description": "Payroll, taxation and compliance" }
```

```
{
  "success": true,
  "message": "Department created successfully.",
  "data": {
    "id": 9,
    "departmentCode": "DEP0009",
    "departmentName": "Finance",
    "description": "Payroll, taxation and compliance",
    "active": true
  },
  "timestamp": "2026-09-21T11:04:22.918233"
}
```

5.2 Get / list departments

GET /api/v1/departments/{id} → 200 "Department fetched successfully." , 404 if absent.

GET /api/v1/departments?

page=0&size=10&search=eng&sortBy=departmentName&sortDirection=asc → 200

"Departments fetched successfully."

5.3 Update Department

Item	Value
Method / path	PUT /api/v1/departments/{id}
Request	UpdateDepartmentRequest (§4.1) — send active
Response	200 "Department updated successfully."
Message	—

Duplicate rule: the duplicate check is skipped when the incoming name `equalsIgnoreCase` the stored name (so fixing the casing of the *same* department is allowed). It runs for a genuinely different name, and it compares against **all** rows including the one being edited.

5.4 Change Department status

Item	Value
Method / path	<code>PATCH /api/v1/departments/{id}/status?active=false</code>
Request body	none
Response	200, <code>data = null</code>
Message	<code>"Department status updated successfully."</code>

`active` is a **required query parameter**, not a body field. Omitting it does **not** give a 400 — `MissingServletRequestParameterException` is not handled explicitly and falls through to the catch-all, so the client sees **500 INTERNAL_SERVER_ERROR**. Always append `?active=`.

5.5 Delete Department

Item	Value
Method / path	<code>DELETE /api/v1/departments/{id}</code>
Response	200, <code>data = null</code> , <code>"Department deleted successfully."</code>
Failure modes	404 if the id does not exist — 500 if employees or designations still reference it

 **Delete warning:** This is a **hard delete** (`repository.delete(entity)`). The service performs **no** dependency check. Because `employees.department_id` and `designations.department_id` are NOT NULL foreign keys, deleting a referenced department throws a `DataIntegrityViolationException` → 500. **Preferred UI flow:** deactivate via the status endpoint; only offer Delete for rows you know are unreferenced.

5.6 Designations (7-12)

Identical shape to departments, with these differences:

- `POST` requires `departmentId` (`@NotNull`). A non-existent `departmentId` → **404 RESOURCE_NOT_FOUND** `"Department not found."`.
- Duplicate rule is scoped per department:
`existsByDesignationNameAndDepartment(name, department)` → 400

"Designation already exists in this department." The same designation name **is** allowed in two different departments.

- On `PUT`, the duplicate check is skipped only when **both** the name matches ignoring case **and** the department id is unchanged.
- Codes: `DES0001`, `DES0002`, ...
- List default sort: `designationName . search` matches `designationName` only.
- Response echoes `departmentId` **and** `departmentName` — use them to render the parent without a second call.
- Messages: `"Designation created successfully."`, `"Designation fetched successfully."`, `"Designations fetched successfully."`, `"Designation updated successfully."`, `"Designation status updated successfully."`, `"Designation deleted successfully."`

POST `/api/v1/designations`

```
{ "designationName": "Software Engineer", "departmentId": 4,  
  "description": "IC role" }
```

5.7 Job titles (13–18)

Differences from the other two:

- `POST` requires `designationId ( @NotNull )` → 404 `"Designation not found."` if it does not exist.
- Duplicate rule is scoped per designation:
`existsByJobTitleAndDesignationId(jobTitle, designationId)` → 400 `"Job title already exists."`
- `PUT` **performs NO duplicate check at all**. Unlike create, update can write a job title that already exists under the same designation. Enforce uniqueness client-side on the edit form (or expect duplicates).
- Codes: `JT0001`, `JT0002`, ... (padded from index 2 of the code, since the prefix is only 2 chars).
- List default sort: `jobTitle . search` matches `jobTitle` only.
- **No `description` field** on job titles.
- Messages: `"Job title created successfully."`, `"Job title fetched successfully."`, `"Job titles fetched successfully."`, `"Job title updated successfully."`, `"Job title status updated successfully."`, `"Job title deleted successfully."`

6. Typical UI flows

6.1 Master data screens (three near-identical CRUD pages)

1. GET /api/v1/{resource}?page=0&size=10&search=&sortBy=<entityProp>&sortDirection=asc
2. Table columns from the response DTO; `active` rendered as a badge with a toggle that calls `PATCH .../status?active=.`
3. "Add" → modal with the create DTO fields; `Code` shown as `"— (auto) "`.
4. "Edit" → prefill from the row, including `active` ; on submit send the **whole** `Update...Request` .
5. "Delete" → confirm dialog, with the dependency warning from §11.2; recommend routing users to Deactivate instead.

6.2 Employee-profile cascading dropdown

1. GET /api/v1/lookups/departments
2. user picks `departmentId`
3. GET /api/v1/lookups/departments/{departmentId}/designations?departmentId={departmentId}
4. user picks `designationId`
5. GET /api/v1/lookups/designations/{designationId}/job-titles?designationId={designationId}

Rules to implement:

- Each child dropdown is **disabled until its parent is chosen**, and must be **cleared and re-fetched** whenever the parent changes (otherwise you can post a designation that does not belong to the selected department — the backend only rejects it in the employee module via `existsByIdAndDepartmentId` , not here).
- Lookups return **active rows only**, sorted by name ascending. An inactive designation will never appear.
- When *editing* an existing employee whose designation/department was later deactivated, the saved value will be missing from the lookup list: render the stored value as a disabled placeholder option instead of blanking the field.

7. Error handling table

Scenario	HTTP	errorCode	Message
Missing / expired / malformed token	401	UNAUTHORIZED	Authentication is required to access this resource.

Scenario	HTTP	errorCode	Message
Role not HR_ADMIN/MANAGER (incl. SUPER_ADMIN)	403	ACCESS_DENIED	You do not have permission to perform this action.
Unknown id on GET/PUT/PATCH/DELETE	404	RESOURCE_NOT_FOUND	Department not found. / Designation not found. / Job title not found.
<code>departmentId</code> / <code>designationId</code> points to a missing parent	404	RESOURCE_NOT_FOUND	Department not found. / Designation not found.
Blank required name, missing parent id	400	VALIDATION_FAILED	must not be blank / must not be null / Job title is required. / Designation is required.
Duplicate name (create, and update when changed)	400	VALIDATION_FAILED	Department already exists. / Designation already exists in this department. / Job title already exists.
<code>active</code> query param missing on <code>/status</code>	500	INTERNAL_SERVER_ERROR	An unexpected error occurred. (MissingServletRequestParameterException: ...)
<code>active=notabool</code>	400	INVALID_REQUEST	Invalid value 'notabool' for parameter 'active'
<code>sortBy</code> is not a JPA property	500	INTERNAL_SERVER_ERROR	An unexpected error occurred. (PropertyReferenceException: ...)
Value longer than the DB column	500	INTERNAL_SERVER_ERROR	DataIntegrityViolation-derived message
Delete referenced by employee/designation/job-title	500	INTERNAL_SERVER_ERROR	DataIntegrityViolation-derived message

Scenario	HTTP	errorCode	Message
<code>size / page</code> not a number	400	INVALID_REQUEST	Invalid value 'x' for parameter 'page'

Because several realistic cases surface as **500**, do not assume "500 = server bug" when wiring error toasts — inspect `errorCode` and `message`, and add client-side guards for the four avoidable ones (missing `active`, bad `sortBy`, over-length text, deleting referenced rows).

8. TypeScript contracts

```

/* ----- Envelopes ----- */
export interface ApiResponse<T> {
  success: boolean;
  message: string;
  data: T;
  timestamp: string; // ISO-8601 LocalDateTime, no timezone offset
}

export interface ApiError {
  success: false;
  message: string;
  errorCode: string;
  path: string;
  timestamp: string;
}

export interface PageResponse<T> {
  content: T[];
  page: number; // 0-based
  size: number;
  totalElements: number;
  totalPages: number;
  first: boolean;
  last: boolean;
}

/* ----- Departments ----- */
export interface DepartmentResponse {
  id: number;

```

```

    departmentCode: string;
    departmentName: string;
    description: string | null;
    active: boolean;
}

export interface CreateDepartmentRequest {
    departmentName: string;
    description?: string | null;
}

export interface UpdateDepartmentRequest {
    departmentName: string;
    description?: string | null;
    active: boolean; // ALWAYS send - omitting it deactivates the row
}

/* ----- Designations ----- */
export interface DesignationResponse {
    id: number;
    designationCode: string;
    designationName: string;
    departmentId: number;
    departmentName: string;
    description: string | null;
    active: boolean;
}

export interface CreateDesignationRequest {
    designationName: string;
    departmentId: number;
    description?: string | null;
}

export interface UpdateDesignationRequest {
    designationName: string;
    departmentId: number;
    description?: string | null;
    active: boolean;
}

/* ----- Job titles ----- */
export interface JobTitleResponse {
    id: number;

```

```

    jobTitleCode: string;
    jobTitle: string;
    designationId: number;
    designationName: string;
    active: boolean;
}

export interface CreateJobTitleRequest {
    jobTitle: string;
    designationId: number;
}

export interface UpdateJobTitleRequest {
    jobTitle: string;
    designationId: number;
    active: boolean;
}

/* ----- Lookups ----- */
export interface LookupResponse { id: number; name: string; }

/* ----- List query ----- */
export interface MasterListQuery {
    page?: number; // default 0
    size?: number; // default 10
    search?: string; // matches the name field only
    sortBy?: string; // entity property — see §11.4 allowlist
    sortDirection?: 'asc' | 'desc'; // default 'asc'
}

```

9. API client examples

```

// api/master.ts
import { ApiResponse, ApiError, PageResponse, MasterListQuery,
    DepartmentResponse, CreateDepartmentRequest, UpdateDepartmentRequest }
    from './types';

const BASE = '/api/v1';

async function request<T>(path: string, init?: RequestInit): Promise<T> {
    const res = await fetch(`${BASE}${path}`, {

```

```

    ...init,
    headers: {
      'Content-Type': 'application/json',
      Authorization: `Bearer ${getAccessToken()}`,
      ...(init?.headers ?? {}),
    },
  });

const body = await res.json().catch(() => null);

if (!res.ok) {
  const error = body as ApiError;
  throw Object.assign(new Error(error?.message ?? `HTTP ${res.status}`),
    {
      status: res.status,
      errorCode: error?.errorCode,
    });
}
return (body as ApiResponse<T>).data;
}

export const departments = {
  list: (q: MasterListQuery = {}) =>
    request<PageResponse<DepartmentResponse>>(
      '/departments?' + new URLSearchParams({
        page: String(q.page ?? 0),
        size: String(q.size ?? 10),
        search: q.search ?? '',
        sortBy: q.sortBy ?? 'departmentName',
        sortDirection: q.sortDirection ?? 'asc',
      }),
    ),

  get: (id: number) => request<DepartmentResponse>(`/departments/${id}`),

  create: (body: CreateDepartmentRequest) =>
    request<DepartmentResponse>('/departments', {
      method: 'POST', body: JSON.stringify(body),
    }),

  // NOTE: pass the whole form state back, including `active`.
  update: (id: number, body: UpdateDepartmentRequest) =>
    request<DepartmentResponse>(`/departments/${id}`, {
      method: 'PUT', body: JSON.stringify(body),
    }),

```

```

setStatus: (id: number, active: boolean) =>
  request<void>(`/departments/${id}/status?active=${active}`, { method: 'PUT' });

remove: (id: number) => request<void>(`/departments/${id}`, { method: 'DELETE' });

// Axios variant for the trickier lookup paths (query param is mandatory)
import axios from 'axios';

export const lookups = {
  departments: () =>
    axios.get<ApiResponse<LookupResponse[]>>(`/api/v1/lookups/departments`),
    .then(r => r.data.data),

  designationsOf: (departmentId: number) =>
    axios.get<ApiResponse<LookupResponse[]>>(
      `/api/v1/lookups/departments/${departmentId}/designations`,
      { params: { departmentId } }, // <-- required, see $11
    ).then(r => r.data.data),

  jobTitlesOf: (designationId: number) =>
    axios.get<ApiResponse<LookupResponse[]>>(
      `/api/v1/lookups/designations/${designationId}/job-titles`,
      { params: { designationId } }, // <-- required
    ).then(r => r.data.data),
};

```

10. Client-side validation helper

Mirror the server rules so users never see a 500:

```

const MAX = { departmentName: 100, description: 255, designationName: 255 };

export function validateDepartment(form: { departmentName: string; description: string }) {
  const errors: Record<string, string> = {};
  const name = form.departmentName?.trim() ?? '';
  if (!name) errors.departmentName = 'Department name is required.';
  else if (name.length > MAX.departmentName) errors.departmentName = 'Max 100 characters.';
  if ((form.description?.length ?? 0) > MAX.description)
    errors.description = 'Max 255 characters.';
}

```

```

    return errors;
}

export function validateDesignation(form: { designationName: string; departmentId: string; }) {
    const errors: Record<string, string> = {};
    if (!form.designationName?.trim()) errors.designationName = 'Designation name is required.';
    else if (form.designationName.trim().length > MAX.designationName)
        errors.designationName = 'Max 255 characters.';
    if (!form.departmentId) errors.departmentId = 'Department is required.';
    return errors;
}

export function validateJobTitle(form: { jobTitle: string; designationId?: string; }) {
    const errors: Record<string, string> = {};
    if (!form.jobTitle?.trim()) errors.jobTitle = 'Job title is required.';
    else if (form.jobTitle.trim().length > MAX.jobTitle) errors.jobTitle = 'Job title is too long.';
    if (!form.designationId) errors.designationId = 'Designation is required.';
    return errors;
}

```

Also do a **case-insensitive** duplicate warning in the UI (the server check is exact-match only – §11.3), and always send `active` explicitly on update.

11. Backend quirks, defects and gotchas

These were all verified against the code. Handle them on the frontend; they are documented here because they are **not** things you can fix from the client.

11.1 Lookup endpoints bind the id as a query parameter, not as a path variable

`LookupController` declares:

```

@GetMapping("/departments/{departmentId}/designations")
public ... getDesignations(@RequestParam Long departmentId) { ... } //

```

The `{departmentId}` in the path is decorative. Calling

`/api/v1/lookups/departments/4/designations` **without** `?departmentId=4` throws

`MissingServletRequestParameterException`, which is unhandled → **500**. Always send both the path segment and the query parameter (the URL in §3/§7 is the correct form).

11.2 Delete is a hard delete with no dependency guard

No service checks whether a department/designation/job title is referenced by employees or child rows. `employees` references all three with `NOT NULL` FKs, and `designations` / `job_titles` reference their parents. Deleting a used row → **500**. Recommended UI: hide/soften the Delete action for referenced rows and offer **Deactivate** (the `/status` endpoint) as the primary action.

11.3 Duplicate detection is case- and whitespace-sensitive, and only on the name

The database is PostgreSQL, so `existsByDepartmentName("finance")` does **not** find `"Finance"`. Consequences:

- `"IT"`, `"it"` and `"It"` can all be created in the same table.
- `"Finance "` (trailing space) also passes, because the server never trims.

Normalise (trim + collapse internal whitespace) before sending, and run a case-insensitive duplicate check against the loaded page in the UI.

11.4 `sortBy` must be a JPA entity property, and bad values cause 500

Allowlist the values you send; anything else throws `PropertyReferenceException` → 500.

Resource	Valid <code>sortBy</code> values
Departments	<code>id</code> , <code>departmentCode</code> , <code>departmentName</code> , <code>description</code> , <code>active</code> , <code>createdAt</code> , <code>updatedAt</code>
Designations	<code>id</code> , <code>designationCode</code> , <code>designationName</code> , <code>description</code> , <code>active</code> , <code>createdAt</code> , <code>updatedAt</code> , <code>department.id</code> (use with care – prefer client-side sorting by <code>departmentName</code>)
Job titles	<code>id</code> , <code>jobTitleCode</code> , <code>jobTitle</code> , <code>active</code> , <code>createdAt</code> , <code>updatedAt</code> , <code>designation.id</code>

Sorting by the response-only field `departmentName` on designations is **not** supported server-side (it is an association property), so sort that column in the client or omit it.

11.5 `sortDirection` fails open

Only `"desc"` (case-insensitive) is special-cased; every other value — including typos like `"down"` — quietly sorts ascending with no error. Send only `asc` / `desc`.

11.6 `PUT` silently deactivates when `active` is omitted

`Update...Request.active` is a primitive `boolean` with no default, so a missing field deserialises to `false`. Since update also writes `active` straight onto the entity, omitting it turns the record off while returning `200` and a message that says "updated successfully". Always echo `active` back from the loaded row.

11.7 Job-title update skips the uniqueness check

`JobTitleServiceImpl.update()` never calls `existsByJobTitleAndDesignationId`, unlike `create()`. Editing a job title into a name that already exists under the same designation is accepted. Guard this in the edit form.

11.8 `SUPER_ADMIN` is denied by the `@PreAuthorize` expressions

Every method uses `hasAnyRole('HR_ADMIN', 'MANAGER')`. The Swagger tags promise `SUPER_ADMIN` access, but the annotation does not grant it. If a `SUPER_ADMIN` must manage master data, that is a backend change — do not advertise the screen to them as working today.

11.9 Job-title controller has a malformed class-level `@PreAuthorize`

`@PreAuthorize("hasAnyRole('SUPER_ADMIN, MANAGER, HR_ADMIN')")` on the class is a single role string (commas inside one quote) and would match nobody. It is harmless only because **every method** in that controller carries its own method-level `@PreAuthorize`, which takes precedence. Do not rely on the class-level expression, and do not remove the method-level ones.

11.10 Other small notes

- `POST` returns **201**, all other success responses return **200**. Do not key success off `=== 200` only.
- `data` is `null` on delete/status-change — type them as `void` and ignore the body.
- Timestamps come from `LocalDateTime` with **no timezone**; treat them as server-local when formatting.

- `search` is not a full-text search: no code/description matching, no multi-word AND, no fuzzy matching. If you need those, filter client-side over the fetched page.
 - `size` has **no server-side cap** — keep it ≤ 100 to protect the UI and the connection pool (Hikari is configured for 8 max connections in `application-dev/prod.properties`).
 - There is no optimistic-locking token on these entities (no `@Version` on `BaseEntity`), so two admins editing the same row last-write-wins silently.
 - The seeder inserts a baseline department from `DataInitializer`, so a brand-new environment already has at least one row; do not assume an empty table when writing "first department" onboarding copy.
-

Frontend rules to remember

1. **Always send `active` on `PUT`.** Omitting it can silently deactivate the record.
2. **Always send lookup IDs twice.** For dependent lookups, the ID is required both in the path and as a query parameter.
3. **Prefer Deactivate over Delete.** Deleting referenced master data can surface as a `500`.
4. **Keep dropdowns cascading.** Clear and reload child values whenever the parent changes.
5. **Validate before calling the API.** Several avoidable input problems currently surface as `500` responses.

12. Integration checklist

- ☐ Attach the Bearer token to all 21 endpoints; treat 401/403 distinctly in the toast copy.
- ☐ Use `PATCH .../status?active=<boolean>` as a **query parameter**, never a body.
- ☐ Always send `active` in every `PUT` body.
- ☐ Cap `size` at ≤ 100 ; `page` is 0-based.
- ☐ Allowlist `sortBy`; send `sortDirection` as only `asc` / `desc`.
- ☐ Enforce the DB length limits client-side.
- ☐ Trim + case-insensitively check duplicates before create/update.
- ☐ Block duplicate job-title names in the edit form (server will not).
- ☐ Route deletion through a dependency warning; prefer deactivate.
- ☐ For lookups, pass the id **both** in the path and as a query param.
- ☐ Clear child dropdowns whenever a parent changes.
- ☐ Render `data: null` responses without crashing.
- ☐ Verify the role gate: HR_ADMIN and MANAGER only — hide the master-data menu from SUPER_ADMIN/PAYROLL_ADMIN/EMPLOYEE until the backend grants them access.

